

Практическая работа 1. Расширение языка CUDA C

В этой работе мы узнаем, какие расширения получил язык программирования C/C++ для написания кода программ, реализуемых на GPU, и напишем свою первую программу.

Для программирования на CUDA предусмотрены специальные расширения языка C/C++. Они состоят из:

- спецификаторов функций, определяющих, откуда происходит вызов функций и где они исполняются;
- спецификаторов переменных, задающих тип памяти, выделяемой под эти переменные;
- встроенных векторных типов;
- встроенных переменных для задания размера сетки и блока, а также для индексации нитей внутри блока и блоков внутри сетки;
- специальных директив вызова ядра, задающих иерархию нитей.

Далее подробно рассмотрим каждое из этих расширений, но сначала давайте напишем свою первую программу на CUDA.

Первая программа

В любом учебном пособии первая программа, которую предлагают вам написать – это программа, выводящая на консоль «Hello, world!». Данная программа на языке программирования C++ выглядит следующим образом:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>

__global__ void HelloWorld()
{
    printf("Hello world, %d, %d\n", blockIdx.x,
threadIdx.x);
}
```

```

int main()
{
    HelloWorld << <1, 1 >> >();
    // ожидаем нажатия любой клавиши
    getchar();
    return 0;
}

```

Первые две строки в программе (два `#include`) не обязательны. В среде Visual Studio они подключаются по умолчанию.

В отличие от файлов с программами на языке программирования C++, имеющих расширения `.cpp`, файлы с программами, которые используют технологию CUDA, должны иметь расширение `.cu`.

Спецификаторы функций и переменных

В расширении языка C/C++ используют три спецификатора функций, которые определяют, откуда функции вызываются и где выполняются (табл. 3).

Таблица 3

Спецификаторы функций в CUDA

Спецификатор	Функция выполняется на	Функция вызывается из
<code>__global__</code>	device (GPU)	host (CPU)
<code>__device__</code>	device (GPU)	device (GPU)
<code>__host__</code>	host (CPU)	host (CPU)

Спецификатор `__global__` определяет ядро, и соответствующая функция должна возвращать тип `void`. При вызове ядра обязательно указывается конфигурация вызова.

На функции, исполняемые на устройстве (GPU) (спецификаторы `__global__` и `__device__`), накладываются следующие ограничения:

- не поддерживается рекурсия;

- не поддерживаются static переменные внутри функций;
- не поддерживается переменное число аргументов.

Спецификаторы `__host__` и `__device__` можно использовать совместно, это означает, что данная функция может выполняться как на устройстве, так и на хосте. Спецификаторы `__global__` и `__host__` не могут быть использованы совместно.

Адрес функции, помеченной спецификатором `__device__`, нельзя использовать в качестве указателя на функцию, а `__global__` – можно.

Для размещения переменных в памяти устройства (GPU) используются следующие спецификаторы: `__device__`, `__shared__`, `__constant__`. Их основные свойства указаны в табл. 4.

Таблица 4

Спецификаторы переменных в CUDA

Спецификатор	Память GPU	Область видимости	Время жизни
<code>__device__</code>	глобальная	сетка (grid)	программа
<code>__shared__</code>	разделяемая	блок (block)	блок
<code>__constant__</code>	константная	сетка (grid)	программа

На использование введенных переменных накладывается ряд ограничений:

- соответствующие переменные могут быть объявлены только в пределах одного файла, то есть они не могут быть помечены квалификатором `extern`;
- для записи в переменную, помеченную как `__constant__`, должны использоваться специальные CUDA функции;
- переменная `__shared__` не может быть проинициализирована при объявлении;
- указанные спецификаторы не могут применяться к полям структуры.

Встроенные типы данных и встроенные переменные

В CUDA добавлены 1/2/3/4-мерные векторы, образованные из базовых типов языка C:

```
char1, uchar1, char2, uchar2, char3, uchar3, char4,
uchar4;
short1, ushort1, short2, ushort2, short3, ushort3,
short4, ushort4;
int1, uint1, int2, uint2, int3, uint3, int4, uint4;
long1, ulong1, long2, ulong2, long3, ulong3, long4,
ulong4;
float1, float2, float3, float4;
double1, double2.
```

Обращение к компонентам вектора осуществляется по названиям измерений: x, y, z и w, например,

```
uint4 vec;
int x = vec.x;
```

Для этого типа данных не предусмотрены привычные векторные операции, например, нельзя сложить два таких вектора с помощью оператора «+», необходимо использовать покомпонентное сложение. Для задания значений таких переменных предусмотрен оператор `make_<typename>`, например,

```
int2 vec = make_int2(2,5); // создаем вектор с координатами (2,5)
```

Для задания размерности в CUDA добавлен специальный тип `dim3`. Он основан на типе данных `uint3`, но дополнительно снабжен конструктором, который заполняет все не указанные компоненты вектора единицами:

```
dim3 block (16,16); // создали трехмерный вектор
с компонентами (16,16,1)
```

Указанный тип данных удобно использовать при задании сетки для запуска основных вычислительных ядер.

Для того, чтобы ядро могло однозначно определить номер нити и, соответственно, номер элемента массива данных для обработки в CUDA предусмотрены встроенные переменные *threadIdx* и *blockIdx* типа *dim3*. Каждая из этих переменных является трехмерным целочисленным вектором. Размеры сетки (*grid*) могут быть получены ядром через встроенные переменные *blockDim* и *gridDim* также типа *dim3*.

Рассмотрим отображение локального номера нити и локального номера блока в глобальный номер нити. Схематично это отображение представлено на рис. 6.

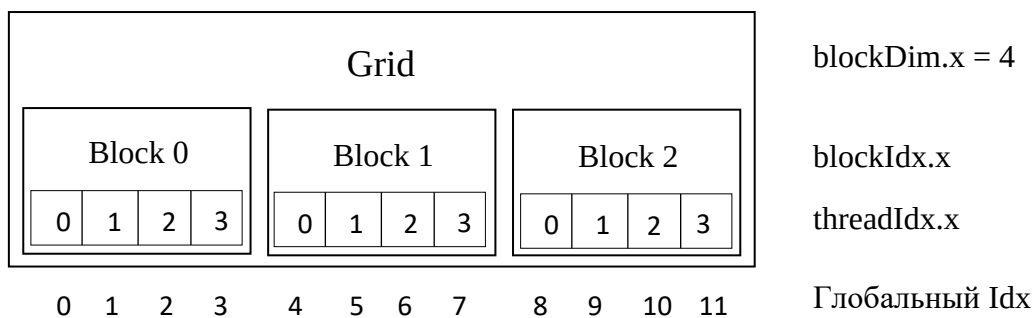


Рис. 6. Отображение локальных индексов нитей в глобальные

Зная номер нити внутри блока и номер блока внутри сетки, а также размерность сетки, можно вычислить глобальный индекс нити:

$$\text{Idx} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

Для получения глобального индекса нити в случае двумерных и трехмерных сеток сначала рекомендуется найти индекс нити относительно каждого из измерений:

$$\text{Idx} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{Idy} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$$

Для двумерного массива данных размера (*DimX*, *DimY*) глобальный индекс нити вычисляется по формуле (см. рис.7):

$$\text{Idx} = \text{Idx} + \text{Idy} * \text{DimX}$$

			0			1			2		
			0	1	2	0	1	2	0	1	2
			0	1	2	3	4	5	6	7	8
			0	1	2	3	4	5	6	7	8
0	0	0					4				
		1	1			11					
		2	2						24		
1	0	3				30					
		1	4								
		2	5								
2	0	6									
		1	7		64						
		2	8								80

Idy

threadIdx.y

blockIdx.y

blockIdx.x

threadIdx.x

Idx = blockIdx.x*blockDim.x+threadIdx.x

11=(0*3+2)+(0*3+1)*9

24=(2*3+0)+(0*3+2)*9

30=(1*3+0)+(1*3+0)*9

80=(2*3+2)+(2*3+2)*9

Id=Idx+Idy*9

Рис. 7. Вычисление глобального номера нити для двумерной сетки

Максимальное число нитей в одном блоке ограничено, так как все они располагаются на одном потоковом процессоре и вынуждены делить между собой ограниченное количество ресурсов. Как правило, число нитей в блоке может достигать 512 или 1024 в зависимости от архитектуры устройства. Однако вычислительное ядро может быть запущено одновременно на большом количестве блоков, поэтому общее количество нитей равно произведению числа нитей в одном блоке на количество блоков.

Директивы вызова ядра

Как мы уже видели, для запуска ядра на GPU в простейшем случае используется конструкция вида:

```
kernel<<<GridDim, BlockDim>>>(args);
```

Здесь *kernel* – название соответствующей `__global__` функции ядра, а *GridDim* и *BlockDim* – переменные типа `dim3`, задающие размерность сетки и блоков внутри сетки.

Размерность сетки (в блоках) и размерность блоков (в нитях) можно задать двумя способами:

```
dim3 grid, block;
grid.x = 2; grid.y = 4;
```

```
block.x = 8; block.y = 16;
```

или

```
dim3 grid(2, 4);  
dim3 block(8,16);
```

Через `args` обозначены аргументы функции ядра, их может быть несколько. Рассмотрим следующий пример:

```
kernel<<<dim3(n/256), dim3(16,16)>>>(n, a);
```

В этом примере происходит вызов функции ядра `kernel` параллельно на `n` нитях (`n` целочисленно делится на 256), потому что количество нитей в одном блоке равно $16 \times 16 = 256$ (двумерный блок), а количество блоков в сетке равно $n/256$ (одномерная сетка блоков), таким образом, общее количество нитей равно `n`. В качестве аргументов функции передаются два параметра – `a` и `n`.

В примере ниже приведен фрагмент кода программы, увеличивающий все элементы массива на заданное число `b`, в реализации для CPU и GPU:

CPU:

```
float *data;  
float b;  
  
// ... выделение памяти под указатель и инициализация значений  
  
for(int i=0; i<n; i++)  
    data[i] = data[i] + b;
```

GPU:

```
__global__ void kernel(float *data, float b){  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    data[idx] = data[idx] + b;  
}
```

Практическая часть

1. Напишите функцию «Hello World» с различными возможными спецификаторами.
2. Реализуйте ядро, увеличивающее элементы двумерного массива на два.

Контрольные вопросы

1. Перчислите расширения языка CUDA C.
2. Как можно вычислить глобальный номер нити для трехмерного массива размера (DimX, DimY, DimZ)?
3. Какие параметры задаются при вызове ядра?
4. Назовите спецификаторы переменных и функций в CUDA.